

**I ARE
PROGRAMMER**



**I MAKE COMPUTER
BEEP BOOP BEEP
BEEP BOOP**

Глава 2. ООП

Ежелев Г. И.

17 июля 2026 г.



Материалы курса можно найти
по ссылке:

<https://github.com/haaroner/ezh239>

Пространства имён

- Все имена всех объектов сгруппированы в пространства имён (namespace). Хорошей аналогией может являться файловая система, где каждая папка - пространство имён, а файл - название объекта.
- Вы можете добавлять свои namespace, чтобы избежать конфликта имён. Это может улучшить читаемость кода.

```
namespace Foo {  
    void f() {  
        std::cout << "Foo" << std::endl;  
    }  
}  
  
namespace Boo {  
    void f() {  
        std::cout << "Boo" << std::endl;  
    }  
}
```

```
int main() {  
    Foo::f();  
    Boo::f();  
  
    return 0;  
}
```

```
using namespace Foo;  
int main() {  
    f();  
    Boo::f();  
  
    return 0;  
}
```

- ▶ ООП - Объектно Ориентированное Программирование, т.е. мы смотрим на программу как на набор объектов, которые взаимодействуют между собой. Зачем это нужно?
- ▶ Мы постоянно хотим делать код более абстрактным, т.е.:
что-то однотипное $\xrightarrow{\text{объединение}}$ универсальное:
 - Несколько переменных $\rightarrow$ массив
 - Повторяющиеся действия $\rightarrow$ функция и т.п. Это принцип DRY - Dont Repeat Yourself

Если повторяющееся поведение сложное: есть и данные(переменные) и действия (функции) - их логично сгруппировать воедино.

Для этого и нужны классы.

- ▶ Примеров с домиками и чертежами не будет
- ▶ Вращение мотора:
 - Данные: у каждого мотора есть два пина управления, текущая скорость, значение энкодера
 - Действия: Задать новую скорость, заблокировать мотор, узнать значение энкодера
 - Вы это обычно реализовывали 4-мя функциями `void m1_run(speed)`, `m2_run(speed)`, ...
 - Логично объединить всё это, **один раз описав логику взаимодействия с мотором** и далее просто указав на каких пинах эти моторы есть.
- ▶ Объект - это программные модели реальных сущностей в мире. У объектов есть данные(поля) и функции(методы). Мы хотим сгруппировать управление мотором в один объект `Motor`.

- ▶ Объекты очень удобно передавать куда-то, например мы можем создать функцию `moveRobot(angle, speed, m1, m2, m3, m4)`, которая будет принимать 4 объекта моторов и через них управлять движением робота
- ▶ Если логика управления мотором изменится - нужно будет менять только код `Motor`, а `moveRobot` - нет (ведь `moveRobot` не думает, как устроен `Motor`, он только отдает команду `motor.set_speed()`)
- ▶ Еще один пример объекта - строка. Она содержит данные: массив символов (`char*`) и методы `.length()`,...

- Класс задает какие характеристики есть у объекта
 - с помощью переменных (полей)



class Cat

name
age
color
owner

объект класса Cat

name = Мурка
age = 5.1
color = рыжий
owner = сосед



- Каждый объект имеет свои значения характеристик
 - значения полей задают состояние объекта

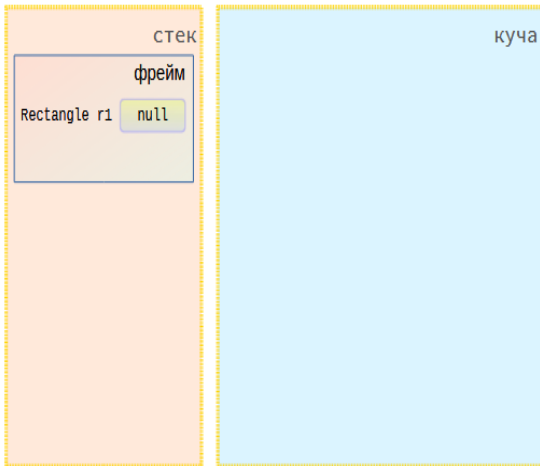


Класс состоит из:

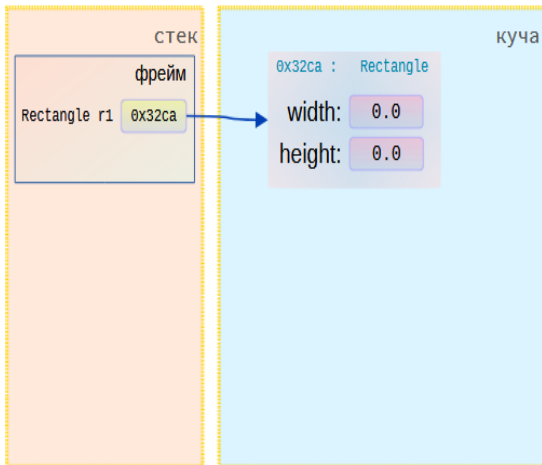
- ▶ Конструктор - метод, вызывающийся при создании нового объекта. Название конструктора совпадает с названием класса.
- ▶ Данные - переменные.
- ▶ Методы - функции класса.
- ▶ Модификаторы доступа - определяющие, кто может использовать данные/методы `public` - кто угодно, `private` - только изнутри класса, `protected` - (?)
- ▶ Например внешним классам нельзя самим опрашивать энкодер, это будет делать сам класс Мотора. А вот прочитать переменную, содержащую положение колеса другие куски программы могут.

- ▶ См. пример для мотора в файле Prog_01.cpp
- ▶ А как объект выглядит в памяти? Разберем на примере класса - Rectangle - геометрическая фигура. (Пример на Java - детали могут отличаться от C++)

```
Rectangle r1;  
r1 = new Rectangle();  
r1.width = 10.5;  
r1.height = 8.2;  
Rectangle r2;  
r2 = r1;  
r1 = new Rectangle();  
r2 = null;
```



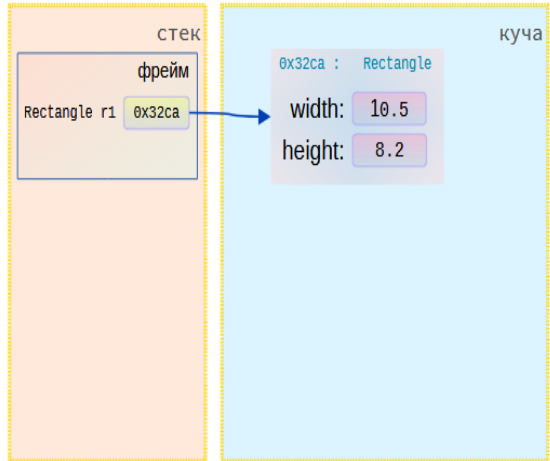
```
Rectangle r1;  
r1 = new Rectangle();  
r1.width = 10.5;  
r1.height = 8.2;  
Rectangle r2;  
r2 = r1;  
r1 = new Rectangle();  
r2 = null;
```



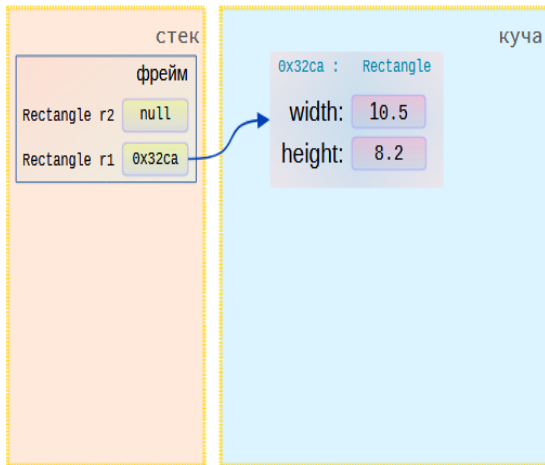
```
Rectangle r1;  
r1 = new Rectangle();  
r1.width = 10.5;  
r1.height = 8.2;  
Rectangle r2;  
r2 = r1;  
r1 = new Rectangle();  
r2 = null;
```



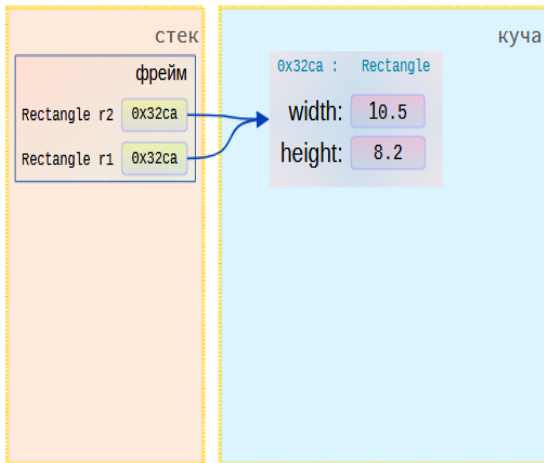
```
Rectangle r1;  
r1 = new Rectangle();  
r1.width = 10.5;  
r1.height = 8.2;  
Rectangle r2;  
r2 = r1;  
r1 = new Rectangle();  
r2 = null;
```



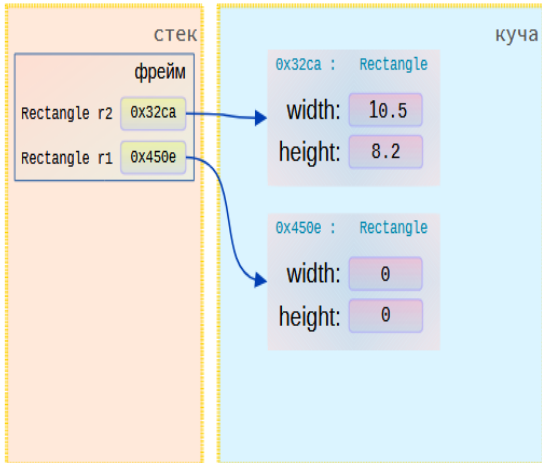
```
Rectangle r1;  
r1 = new Rectangle();  
r1.width = 10.5;  
r1.height = 8.2;  
Rectangle r2;  
r2 = r1;  
r1 = new Rectangle();  
r2 = null;
```



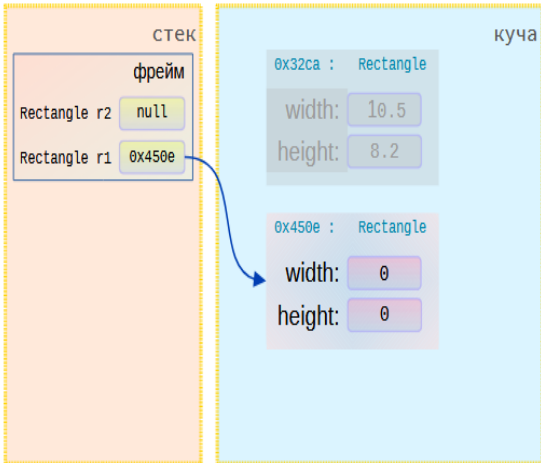
```
Rectangle r1;  
r1 = new Rectangle();  
r1.width = 10.5;  
r1.height = 8.2;  
Rectangle r2;  
r2 = r1;  
r1 = new Rectangle();  
r2 = null;
```




```
Rectangle r1;  
r1 = new Rectangle();  
r1.width = 10.5;  
r1.height = 8.2;  
Rectangle r2;  
r2 = r1;  
r1 = new Rectangle();  
r2 = null;
```



```
Rectangle r1;  
r1 = new Rectangle();  
r1.width = 10.5;  
r1.height = 8.2;  
Rectangle r2;  
r2 = r1;  
r1 = new Rectangle();  
r2 = null;
```



- ▶ Как вы заметили - ссылки на объекты тоже хранятся в стеке (тот самый стек из прошлой главы), а сами их данные нет.
- ▶ Данные и методы относятся к конкретному объекту
- ▶ Можно также сделать методы и данные, относящиеся ко всему классу (т.е. ко всем объектам сразу).
- ▶ Это статический тип - `static`.
- ▶ Если у вас есть что-то внутри и вне класса с одинаковым именем - используйте ссылку `this`. (метод/переменная объекта). Эта ссылка неявно передаётся каждому методу
- ▶ Класс можно рассматривать как свой, кастомный тип данных (как `int`, `char`, `string` и т.п.).

- Объект (instance) имеет собственные значения полей
 - Для разных объектов значения полей разные
 - Это поля экземпляра (не статические)
- Особый вид поля - статическое (static field)
 - Статическое поле - общее для всех объектов класса
 - Его значение - одно для всего класса
 - Статическое поле - поле класса (в Metaspase - часть кучи)



```
class Rectangle {  
    double width;  
    double height;  
    static int numOfSides = 4;  
}
```

```
Rectangle r1 = new Rectangle();  
int x = r1.numOfSides; // 4  
x = Rectangle.numOfSides; // 4
```

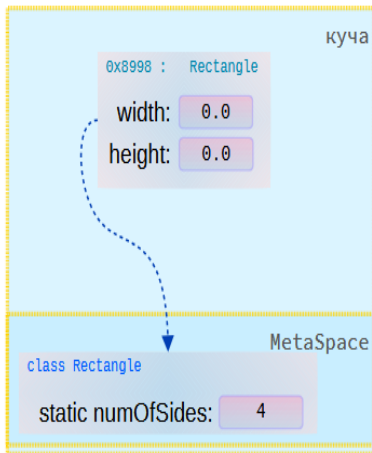
```
class Person {  
    int id;  
    String name;  
    ...  
    static int maxID = 0;  
}
```

```
Person p1 = new Person();  
p1.id = Person.maxID++; // 1  
Person p2 = new Person();  
p2.id = p2.maxID++; // 2
```



```
class Rectangle {  
    double width;  
    double height;  
    static int numOfSides = 4;  
}
```

```
Rectangle r1 = new Rectangle();  
int x = r1.numOfSides; // 4  
x = Rectangle.numOfSides;
```



```
class Rectangle {  
    double this.width;  
    double this.height;  
  
    double area(Rectangle this) {  
        return this.width * this.height;  
    }  
}
```

- Такое отношение между классами называется **композиция**

```
class Rectangle {  
    double width;  
    double height;  
    Point corner;  
}
```

```
class Point {  
    double x;  
    double y;  
}
```

```
Point p1 = new Point();  
p1.x = 30.0;  
p1.y = 22.5;  
Rectangle r1 = new Rectangle();  
r1.width = 5.0;  
r1.height = 7.0;  
r1.corner = p1;
```



- Объекты могут быть элементами массива

```
class Rectangle {  
    double width;  
    double height;  
}
```

```
Rectangle r1 = new Rectangle();  
r1.width = 5.0;  
r1.height = 7.0;  
Rectangle r2 = new Rectangle();  
r2.width = 9.64;  
r2.height = Math.PI;  
Rectangle[] array = { r1, r2 };
```



Задание: Напишите класс покемона

- ▶ Данные: имя, текущий уровень (начиная с 1), health, attackPower, healthInc, attackInc
- ▶ Методы: Конструктор (задает имя, и все статы), levelUp(увеличение статов по заданному инкременту), sayHello (вывести имя+ какую-нибудь фразу), attack(Pokemon& target) - изменение здоровья + лог боя в консоль
- ▶ Сами подумайте, что public, что private
- ▶ Доп1: Сделайте класс битвы, принимающий двух покемонов и крутящий поочерёдные удары в цикле.
- ▶ Здоровье, атаку и уровень лучше сделать приватными (чтобы только сам объект контролировал их). Тогда для их изменения нужны getter и setter функции: getHealth(), setHealth(). Такие функции очень часто используются в реальных проектах